

control_operation

REFERENCE

API consumer

start or stop the RFID radio (or the scanner) on demand

Overview

`control_operation` **starts or stops** a reader subsystem on demand. You pick a `controlType` (`RFID` — the radio inventory engine, or `SCANNER` — the barcode scanner) and an `operation` (`START` or `STOP`). For `RFID`, `START` begins an inventory cycle **using the currently configured operating mode** and the device streams tag reads as `dataEVT` events until you `STOP` (or the operating-mode stop conditions are met). It is a state-changing **control-plane (CTRL) command**, and it is **reversible** — a `START` is undone by a `STOP`.

This is a **CTRL-endpoint command**, published on the **control** endpoint

(`<tenantId>/CTRL/clients/cmnd/<serial>`) and answered only there (not on the MGMT/MDM channel).

Returning a response requires an **active CTRL endpoint** (`config_endpoint`, `epType: CTRL`, `activate: true`). Receiving the tag stream that `START` triggers also requires an **active DATA1 endpoint**

(`<tenantId>/DATA/clients/dataevent/<serial>`); see the [data endpoint topic mapping](#)).

Set the regulatory region first. Before you start an RFID inventory, make sure the reader's regulatory region is configured: the reader does not transmit until it is, and the region is set outside MQTT with 123RFID Desktop or 123RFID Mobile. Use `get_current_region` to verify the current region.

A `START` **executes whatever the operating mode has armed** — including destructive access operations. See the danger callout in [Notes and Caveats](#).

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	STATE-CHANGING (starts / stops the radio or scanner)
Reversibility	REVERSIBLE (a <code>START</code> is undone by a <code>STOP</code>)
Supported Operations	<code>START</code> , <code>STOP</code>
Required Request Fields	<code>command</code> , <code>requestId</code> (<code>ctrlOprPayload</code> is optional in the contract; inside it, <code>controlType</code> and <code>operation</code> are required)
Applies To	Endpoint: Control (CTRL). Reader: RFD40 series, RFD90 series
API Version	<code>v1.1</code> (API Version Reporting)
Related	<code>set_operating_mode</code> , <code>get_operating_mode</code> , <code>get_status</code> , <code>reboot</code>

When to Use

Use `control_operation` to drive the radio (or scanner) on demand:

- **Start an inventory** — `controlType: RFID`, `operation: START` begins reading tags with the current operating mode; the device streams `dataEVT` events over the DATA endpoint.
- **Stop an inventory** — `operation: STOP` halts the radio; the tag stream ceases.
- **Control the scanner** — `controlType: SCANNER` starts / stops the barcode scanner instead of the radio.

Configure *how* the reader reads with `set_operating_mode` **first**, then `START` — `control_operation` itself has no tuning parameters. If the inventory state is uncertain, check it with `get_status` before sending `START`.

Topic Mapping

`control_operation` is the one command that drives **both** planes: the command travels over CTRL, while the device publishes the tag events a `START` triggers over DATA.

Plane	Message	Direction	Topic	Note
CONTROL	Request	App → Reader	<code><tenantId>/CTRL/clients/cmnd/<serial></code>	On the control endpoint. Requires an active CTRL endpoint .
CONTROL	Response	Reader → App	<code><tenantId>/CTRL/clients/resp/<serial></code>	Correlated reply; the in-payload <code>requestId</code> links the twin topics.
DATA	Event	Reader → App	<code><tenantId>/DATA/clients/dataevent/<serial></code>	The <code>dataEVT</code> stream a <code>RFID</code> <code>START</code> triggers. Requires an active DATA endpoint to receive it.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

The command is answered **only on the CTRL endpoint**; the tag stream is published on the `DATA1` endpoint's configured publish topic (`data1event` by default). `DATA2` is currently not supported.

See [Topics and Channels](#).

Key Fields

- `ctrlOprPayload.controlType` (request) — `RFID` (radio inventory engine) or `SCANNER` (barcode scanner).
- `ctrlOprPayload.operation` (request) — `START` (begin) or `STOP` (halt).
- `response.code` (response) — read this to determine success vs failure. The contract requires it; note the reported success-path omission (see [Success without a code](#)). See [Response Codes](#).

Request Payload

The envelope is `command`, `requestId`, and the `ctrlOprPayload` (`controlType` + `operation`). Pick an **operation** below; the Example and Schema update together. Use the **Schema** tab for the full field reference and the **Example** tab for the concrete payload.

Operation

Start inventory

Request payload

Example: RFID START

```
1 {
2   "command" : "control_operation",
3   "requestId" : "abcd1432",
4   "ctrlOprPayload" : {
5     "controlType" : "RFID",
6     "operation" : "START"
7   }
8 }
```

Request payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command	* enum	control_operation	control_operation	Fixed command name (pinned by the schema enum).

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
2	requestId *	string	—	abc123	A unique identifier for the request; echoed back in the response.
3	ctrlOprPayload	object	—	—	The control operation to perform. Optional in the contract; when present, the contract requires both controlType and operation within it.
4	controlType *	enum	RFID SCANNER	RFID	Which reader subsystem to control — RFID (the radio inventory engine) or SCANNER (the barcode scanner). Required within the payload.
5	operation *	enum	START STOP	START	START begins the operation for the selected subsystem; STOP halts it. Required within the payload.

Request Semantics

Modes. `ctrlOprPayload.operation` selects the action for the chosen `controlType`:

- `START` — begin the operation. For `RFID`, this starts an inventory cycle using the current operating mode and streams `dataEVT` events over the DATA endpoint.
- `STOP` — halt the active operation. For `RFID`, the tag stream ceases (a single in-flight read may still be flushed shortly after — see Notes).

A `STOP` when no operation is active returns **code** 12 ("No Radio Operation in Progress") — this is **not a failure**, the device is already idle. A `START` while an inventory is already running returns **code** 11 ("Inventory in progress").

- `SCANNER` uses the same request envelope as `RFID`. `ctrlOprPayload` declares one `controlType` (`RFID` | `SCANNER`) and one `operation` (`START` | `STOP`), both required, with no scanner-specific field and no separate payload — a scanner request differs from an RFID request only in the `controlType` value. Decoded barcodes are declared on the DATA plane, not in this command's response: the `dataEVT` data object carries `barcodeData[]` (`decodedBarcode`, plus `symbology`, the barcode type as a hexadecimal string such as `0x03`) alongside `tagData[]`. Codes `11` and `12` above are radio-scoped — the contract describes them as an RFID inventory already running and no radio operation active — and the schema defines no scanner-specific response code, leaving code `23` for a `controlType` or `operation` outside its enum. Physical-trigger behavior and per-model scanner coverage are outside the schema and are not documented here — see Scan Barcodes.

Response Payload

`control_operation` returns a **status-only** envelope — `command`, `requestId`, `apiVersion`, and `response`. The tag reads arrive separately as `dataEVT` events on the DATA endpoint; the response carries no data echo.

Response payload

```
1 {
2   "command": "control_operation",
3   "requestId": "abc123",
4   "apiVersion": "V1.1",
5   "response": {
6     "code": 0,
7     "description": "Success"
8   }
9 }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command * and	string	—	control_operation	Echoes the executed command.
2	requestId * d	string	—	abc123	Echoes the request's requestId — pairs reply to request.
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1	API version reported by this command. Emitted per-handler, not device-wide.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
4	response *	object	—	—	Status-only result — control_operation echoes no configuration.
5	code *	integer	0-101	0	0 = Success; non-zero = error. Required by the contract. Readers have been reported to omit it on success — see Response Payload and Response Codes below.
6	description *	string	—	Success	Human-readable status.

The contract requires `response.code`. Readers have been reported to omit it on a successful reply, returning just `response: { "description": "Success" }` — treat that as success too. See [Success without a code](#).

As reported, a successful `START` or `STOP` can arrive with `code` omitted. This form does not validate against the contract; the reply with `code: 0` above is the documented form:

Response — success reported without code (not accepted by the contract)

```

1  {
2    "command" : "control_operation",
3    "requestId" : "abc123",
4    "apiVersion" : "V1.1",
5    "response" : {
6      "description" : "Success"
7    }
8  }
```

Response Codes

`control_operation` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — the radio (or scanner) started or stopped. (Readers have been reported to omit <code>code</code> on this reply — see above.)

Code	Description
11	Inventory in progress (<code>IOT_ERROR_INVENTORY_IN_PROGRESS</code>) — a <code>START</code> was sent while an RFID inventory is already running. Stop the current inventory first; if the state is uncertain, check <code>get_status</code> .
12	No Radio Operation in Progress (<code>IOT_ERROR_NO_RADIO_OP_IN_PROGRESS</code>) — a <code>STOP</code> was sent while the radio is already idle. Not a failure — no action required. Unique to this command.
23	Invalid enum value (<code>IOT_ERROR_INVALID_ENUM</code>) — <code>controlType</code> or <code>operation</code> is outside its allowed set.
101	Default error — generic catch-all.

Examples

Publish `control_operation` over the CTRL endpoint; the device replies with the status envelope under [Response Payload](#). The **Operation** dropdown under [Request Payload](#) shows the `START` and `STOP` requests. The examples below show what the dropdown does not: the tag stream that `START` triggers, and the two common error responses.

A `RFID` `START` triggers a `dataEVT` **stream** on the DATA endpoint. Each event's `type` echoes the active operating-mode profile, and the mode's `tagMetaDataToEnable` flags gate which `tagData` fields appear (here the default mode reports `EPCid` / `eventNum` / `peakRssi` / `seenCount` only):

DATA plane — dataEVT triggered by START (abridged)

```

1      {
2        "type": "BALANCED_PERFORMANCE" ,
3        "timestamp": "2026-06-16T06:03:36.894Z" ,
4        "data": {
5          "tagData": [
6            {
7              "EPCid": "E28011B0A506007D3DD07C0F" ,
8              "eventNum": 41 ,
9              "peakRssi": -73 ,
10             "seenCount": 256
11           },
12           {
13             "EPCid": "311100000000000000000000" ,
14             "eventNum": 42 ,
15             "peakRssi": -50 ,
16             "seenCount": 256
17           },
18           {
19             "EPCid": "11113333880600890630DBEC" ,
20             "eventNum": 43 ,
21             "peakRssi": -53 ,
22             "seenCount": 256
23           },
24           {
25             "EPCid": "000011C1A5000062F7952926" ,
26             "eventNum": 44 ,
27             "peakRssi": -65 ,
28             "seenCount": 256
29           },
30           {
31             "EPCid": "E200001B880600870630DBEB" ,
32             "eventNum": 45 ,
33             "peakRssi": -44 ,
34             "seenCount": 256
35           },
36           {
37             "EPCid": "1111333300004001B043ACB2" ,
38             "eventNum": 46 ,
39             "peakRssi": -36 ,
40             "seenCount": 256
41           },
42           {
43             "EPCid": "11113333333344445555AAAA" ,
44             "eventNum": 47 ,
45             "peakRssi": -32 ,
46             "seenCount": 256
47           },
48           {
49             "EPCid": "11113333A5000062F798E5B6" ,
50             "eventNum": 48 ,
51             "peakRssi": -34 ,
52             "seenCount": 256
53           },
54           {

```

```

55     "EPCid" : "11113333333344445555AAAA" ,
56     "eventNum" : 49 ,
57     "peakRssi" : -36 ,
58     "seenCount" : 256
59 },
60 {
61     "EPCid" : "11115555222233334444A9B6" ,
62     "eventNum" : 50 ,
63     "peakRssi" : -43 ,
64     "seenCount" : 256
65 },
66 {
67     "EPCid" : "111133332222333344445555" ,
68     "eventNum" : 51 ,
69     "peakRssi" : -44 ,
70     "seenCount" : 256
71 },
72 {
73     "EPCid" : "111133332222333344445550" ,
74     "eventNum" : 52 ,
75     "peakRssi" : -43 ,
76     "seenCount" : 256
77 },
78 {
79     "EPCid" : "FFFFFFFF3435363742426666" ,
80     "eventNum" : 53 ,
81     "peakRssi" : -51 ,
82     "seenCount" : 256
83 },
84 {
85     "EPCid" : "11113333333344445555AAAA" ,
86     "eventNum" : 54 ,
87     "peakRssi" : -46 ,
88     "seenCount" : 256
89 },
90 {
91     "EPCid" : "11113333880600900630DBF1" ,
92     "eventNum" : 55 ,
93     "peakRssi" : -52 ,
94     "seenCount" : 256
95 },
96 {
97     "EPCid" : "E200001B880600840630DBE2" ,
98     "eventNum" : 56 ,
99     "peakRssi" : -57 ,
100     "seenCount" : 256
101 },
102 {
103     "EPCid" : "111133332222333344445555" ,
104     "eventNum" : 57 ,
105     "peakRssi" : -63 ,
106     "seenCount" : 256
107 },
108 {
109     "EPCid" : "111133332222333344445555" ,
110     "eventNum" : 58 ,

```

```

111         "peakRssi" : -63 ,
112         "seenCount" : 256
113     },
114     {
115         "EPCid" : "11113333222333344445555" ,
116         "eventNum" : 59 ,
117         "peakRssi" : -62 ,
118         "seenCount" : 256
119     },
120     {
121         "EPCid" : "E200001B880600870630DBEB" ,
122         "eventNum" : 60 ,
123         "peakRssi" : -44 ,
124         "seenCount" : 256
125     },
126     {
127         "EPCid" : "000011C1A5000062F7952926" ,
128         "eventNum" : 61 ,
129         "peakRssi" : -66 ,
130         "seenCount" : 256
131     },
132     {
133         "EPCid" : "11113333B22FB22FB22FB22F" ,
134         "eventNum" : 62 ,
135         "peakRssi" : -65 ,
136         "seenCount" : 256
137     },
138     {
139         "EPCid" : "AAAABBBB11112222CCCCDDDD" ,
140         "eventNum" : 63 ,
141         "peakRssi" : -60 ,
142         "seenCount" : 256
143     },
144     {
145         "EPCid" : "BEDD11112222333344445555" ,
146         "eventNum" : 64 ,
147         "peakRssi" : -45 ,
148         "seenCount" : 256
149     },
150     {
151         "EPCid" : "311100000000000000000000" ,
152         "eventNum" : 65 ,
153         "peakRssi" : -51 ,
154         "seenCount" : 256
155     },
156     {
157         "EPCid" : "11113333880600890630DBEC" ,
158         "eventNum" : 66 ,
159         "peakRssi" : -53 ,
160         "seenCount" : 256
161     },
162     {
163         "EPCid" : "1111FB63AC1F3841DADA00000000CAFEDADACAFEDADACAFE00001111" ,
164         "eventNum" : 67 ,
165         "peakRssi" : -57 ,
166         "seenCount" : 256

```

```

167     },
168     {
169         "EPCid" : "E28011B0A506007D3DD07D01" ,
170         "eventNum" : 68 ,
171         "peakRssi" : -72 ,
172         "seenCount" : 256
173     },
174     {
175         "EPCid" : "E28011B0A506007D3DD07C0F" ,
176         "eventNum" : 69 ,
177         "peakRssi" : -69 ,
178         "seenCount" : 256
179     },
180     {
181         "EPCid" : "E28011B0A506007D3DD07D00" ,
182         "eventNum" : 70 ,
183         "peakRssi" : -70 ,
184         "seenCount" : 256
185     },
186     {
187         "EPCid" : "11115555222233334444A9B6" ,
188         "eventNum" : 71 ,
189         "peakRssi" : -41 ,
190         "seenCount" : 256
191     },
192     {
193         "EPCid" : "111133332222333344445550" ,
194         "eventNum" : 72 ,
195         "peakRssi" : -45 ,
196         "seenCount" : 256
197     },
198     {
199         "EPCid" : "311100000000000000000000" ,
200         "eventNum" : 73 ,
201         "peakRssi" : -65 ,
202         "seenCount" : 256
203     },
204     {
205         "EPCid" : "111133332222333344445555" ,
206         "eventNum" : 74 ,
207         "peakRssi" : -61 ,
208         "seenCount" : 256
209     },
210     {
211         "EPCid" : "A1B2222BBBB" ,
212         "eventNum" : 75 ,
213         "peakRssi" : -65 ,
214         "seenCount" : 256
215     }
216 ]
217 }
218 }

```

A **STOP** issued while the radio is **already idle** returns **code 12** (not a failure):

Response — STOP while idle (code 12)

```
1  {
2    "command" : "control_operation" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.1" ,
5    "response" : {
6      "code" : 12 ,
7      "description" : "No Radio Operation in Progress"
8    }
9  }
```

A `START` issued while an inventory is already running returns code 11:

Response — START while inventory running (code 11)

```
1  {
2    "command" : "control_operation" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.1" ,
5    "response" : {
6      "code" : 11 ,
7      "description" : "Inventory in progress"
8    }
9  }
```

Notes and Caveats

⚠️ ARMED ACCESS OPERATIONS EXECUTE ON `START`

`control_operation` has no tuning parameters; a `START` runs whatever `set_operating_mode` last configured. If destructive access operations (`WRITE` / `LOCK` / `KILL`) are **armed**, `START` executes them on every tag in range — `KILL` and `PERMANENT_LOCK` are **irreversible**. Confirm the operating mode is read-only (no armed `accessOperations`) before starting a production inventory. See `set_operating_mode` \$. [Notes and Caveats](#).

- `START` drives two planes; the tag stream requires a **DATA** endpoint. The command round-trip is on the CTRL endpoint, but the tag reads a `RFID` `START` produces are published as `dataEVT` events on the **DATA** endpoint (`<tenantId>/DATA/clients/dataevent/<serial>` for `DATA1_EP`). To receive them, a `DATA1` endpoint must be active — activate one with `config_endpoint`. `dataEVT.type` echoes the active operating-mode profile, and the mode's `tagMetaDataToEnable` flags gate which `tagData` fields appear (`EPCid` is always present).
- `STOP` is immediate at the radio, but a trailing read may still flush. A `STOP` stops the radio right away, yet a single in-flight / buffered read may still be delivered on the DATA plane a couple of seconds after the `STOP` (≈ 2.9 s observed on RFD40) before the stream fully ceases. Do not treat one trailing `dataEVT` after a `STOP` as continued inventory.
- **Configure the mode before you `START`**. Set profiles, power, triggers, `select` filters, and access operations with `set_operating_mode` first, then `START`; `control_operation` itself sets none of these. The operating

mode is **volatile**: it resets to the default (`BALANCED_PERFORMANCE` , no armed access operations) on `reboot` , so re-apply it after a reboot before starting.

- **An attached terminal has priority.** Apps on a terminal attached to the reader, whether Android, iOS or Windows, always have priority over MQTT commands, and a user can start an inventory from the terminal while the reader is connected to the broker. What the reader replies to a `control_operation` that a terminal app overrides is not specified in the available sources. See [The Reader Is Its Own MQTT Client](#).

`control_operation` is **reversible** (`START` ↔ `STOP`) and carries **no credentials** — the tag `EPCid` values in the resulting `dataEVT` stream are identifiers read off physical tags, not secrets. `SCANNER` control follows the same request/response shape as `RFID` (verified paths are `RFID` `START` / `STOP` on RFD40). For the concept-level view, see Start and Stop Inventory.

Related:  [set_operating_mode](#) ·  [get_operating_mode](#) ·  [get_status](#) ·  [dataEVT](#) ·  Start and Stop Inventory ·  Scan Barcodes ·  [Response Codes](#)